

Custom Laser Profile Injection

Zheyuan Chen
York Plasma Institute, University of York

September 11, 2026

This document describes the modifications made to EPOCH's `laser` block to support loading arbitrary laser amplitude and phase profiles from raw binary data files¹. See the EPOCH documentation² for the standard `laser` block parameters that are not covered here.

The custom profile injection feature allows a laser field of the form $E = A \cdot \sin(\omega t + \varphi)$ — with A and φ arbitrary functions of the transverse boundary coordinates and (optionally) time — to be driven entirely from externally generated data files, bypassing EPOCH's built-in analytic profile functions. This is particularly useful when interfacing with external beam propagation codes such as LASY³, or when the target profile cannot be expressed as one of EPOCH's analytic primitives. A Python wrapper for LASY written by the same author can be found here⁴.

The feature is implemented in `epoch2d` and `epoch3d`. `epoch1d` is unmodified.

Two modes exist, selected per laser block:

- **Spatiotemporal** (`use_spatiotemporal_profile = T`): the full amplitude $A(\text{transverse}, t)$ is interpolated from file at every time step. In 2D the data is $A(y, t)$; in 3D it is $A(\text{tr1}, \text{tr2}, t)$.
- **Static spatial** (`use_spatiotemporal_profile = F`): a time-independent spatial profile is interpolated onto the boundary once at start-up; the temporal envelope is set analytically in the deck via `t_profile`.

In both modes the phase can also be read from file with `use_phase_from_file = T`, on the same grid as the amplitude.

Note on defaults: when `use_custom_profile = T` and `use_spatiotemporal_profile` is not given, `epoch2d` defaults to the spatiotemporal mode (T) while `epoch3d` defaults to the static spatial mode (F). Setting the flag explicitly is recommended.

1 EPOCH2D

An example 2D `laser` block using all custom profile features:

```
begin:laser
  boundary = x_min
  intensity_w_cm2 = 1e22
  lambda = 1.0 * micron
  pol = 0

  use_custom_profile = T
```

¹https://epochpic.github.io/documentation/input_deck/binary_files.html

²https://epochpic.github.io/documentation/input_deck/input_deck_laser.html

³<https://github.com/LASY-org/lasy>

⁴<https://github.com/ZheyuanChen/AELP/tree/main>

```

use_spatiotemporal_profile = T
profile_data_file = amplitude.dat

n_t = 50
n_y = 30
y_min = -10.0 * micron
y_max = 10.0 * micron
t_start = 0.0
t_end = 30e-15

use_phase_from_file = T
phase_data_file = phase.dat
end:laser

```

A 2D custom-profile laser block accepts the following parameters:

use_custom_profile Enable custom profile injection. Must be set to T to activate any of the parameters below. The default is F, which leaves all standard EPOCH profile behaviour unchanged.

use_spatiotemporal_profile When T, the full two-dimensional $A(y, t)$ amplitude is read from a raw binary file and re-interpolated at every time step. When F, a one-dimensional static spatial profile $A(y)$ is read (also raw binary) and interpolated onto the boundary once at start-up; the temporal envelope is then set analytically via **t_profile**. Defaults to T in epoch2d.

profile_data_file Path to the amplitude file. Relative paths resolve from the deck's data directory; absolute paths are used as-is. Defaults to **temporal_spatial_profile.dat** (spatiotemporal) or **spatial_profile.dat** (static) if omitted.

n_t_points / **n_t** Number of time points in the profile array. Mandatory when **use_spatiotemporal_profile** = T.

n_transverse_points / **n_y** Number of transverse spatial points in the profile array (≥ 2). Mandatory whenever **use_custom_profile** = T (both modes).

profile_transverse_min / **y_min** Minimum transverse coordinate of the profile grid (metres). Mandatory whenever **use_custom_profile** = T. (For a laser on **y_min/y_max** the transverse axis is physically x ; the **y_min** alias name is kept for backward compatibility, but the boundary-agnostic **profile_transverse_min** reads better there.)

profile_transverse_max / **y_max** Maximum transverse coordinate of the profile grid (metres). Mandatory whenever **use_custom_profile** = T.

t_start, **t_end** Start and end times of the profile grid (seconds). These reuse EPOCH's standard laser elements; the profile time axis runs from **t_start** to **t_end** with **n_t** uniformly spaced points. Outside this window the laser is inactive.

use_phase_from_file When T, the phase is also read from file: on the spatiotemporal path $\varphi(y, t)$ is re-interpolated each time step; on the static path $\varphi(y)$ is interpolated once at start-up. Any **phase** = ... expression in the deck is ignored. Defaults to F, in which case the phase is set as normal via the **phase** deck element.

phase_data_file Path to the raw binary phase file (radians). Follows the same path resolution as **profile_data_file**. The phase file must lie on the same grid as the amplitude file (same **n_t**, **n_y**, bounds, **t_start**, **t_end**). Defaults to **phase_profile.dat** if omitted.

2 EPOCH3D

In 3D the boundary plane has two transverse axes, so the grid declaration uses two transverse point counts and two pairs of bounds. The boundary-agnostic names are `tr1` and `tr2`, whose physical meaning depends on the boundary the laser is attached to:

Boundary	tr1 axis	tr2 axis
<code>x_min/x_max</code>	y	z
<code>y_min/y_max</code>	x	z
<code>z_min/z_max</code>	x	y

An example 3D laser block (spatiotemporal, amplitude + phase, on `x_min`, written with the axis-named aliases):

```
begin:laser
  boundary = x_min
  intensity_w_cm2 = 1e22
  lambda = 1.0 * micron
  pol = 0

  use_custom_profile = T
  use_spatiotemporal_profile = T
  profile_data_file = amplitude.dat

  n_y_points = 48
  n_z_points = 40
  n_t_points = 16
  y_min = -8.0 * micron
  y_max = 8.0 * micron
  z_min = -8.0 * micron
  z_max = 8.0 * micron
  t_start = 0.0
  t_end = 60e-15

  use_phase_from_file = T
  phase_data_file = phase.dat
end:laser
```

The same block written with the boundary-agnostic names (useful for decks generated by boundary-independent tooling):

```
n_tr1 = 48
n_tr2 = 40
n_t = 16
tr1_min = -8.0 * micron
tr1_max = 8.0 * micron
tr2_min = -8.0 * micron
tr2_max = 8.0 * micron
```

A 3D custom-profile laser block accepts the following parameters (in addition to `use_custom_profile`, `use_spatiotemporal_profile`, `profile_data_file`, `use_phase_from_file`, `phase_data_file`, `t_start` and `t_end`, which behave exactly as in 2D):

`n_transverse1_points / n_tr1` Number of points along the first transverse axis (see the table above). Mandatory (≥ 2) whenever `use_custom_profile = T`.

`n_transverse2_points / n_tr2` Number of points along the second transverse axis. Mandatory (≥ 2) whenever `use_custom_profile = T`.

`n_t_points / n_t` Number of time points. Mandatory (≥ 2) when `use_spatiotemporal_profile = T`; ignored on the static spatial path.

`profile_transverse1_min / tr1_min, profile_transverse1_max / tr1_max` Bounds of the profile grid along the first transverse axis (metres). Mandatory whenever `use_custom_profile = T`.

`profile_transverse2_min / tr2_min, profile_transverse2_max / tr2_max` Bounds along the second transverse axis (metres). Mandatory whenever `use_custom_profile = T`.

`n_{x,y,z}_points / n_{x,y,z}, {x,y,z}_min, {x,y,z}_max` Axis-named aliases for the point counts and bounds above. These are resolved to `tr1` or `tr2` according to the boundary table, so a deck for an `x_min` laser can say `n_y_points/n_z_points` and `y_min...z_max` naturally. Naming the propagation axis (e.g. `n_x_points` on an `x_min` laser) is an error — EPOCH reports the offending element and line and suggests the transverse or boundary-agnostic names.

As in epoch2d, the static spatial path uses the raw binary format: the phase plane is interpolated onto the boundary once at start-up alongside the amplitude.

3 File Format

All data files are raw binary with no embedded header. Shape and grid bounds are declared entirely in the deck (see above). This matches EPOCH’s documented binary-file convention used elsewhere in the code (e.g. `particles_from_file`). EPOCH checks the file size against the declared shape at load time and aborts with the expected and actual byte counts if they disagree. Note that the size check cannot detect a transposed array of the same total size — the axis ordering below must be respected.

Each file contains an array of 64-bit IEEE 754 floating point values (the same precision as EPOCH’s `REAL(num)` type), written in Fortran-compatible column-major order:

- **2D spatiotemporal** $A(y, t)$: transverse y fastest-varying, time slowest. $n_y \times n_t$ values.
- **2D static spatial** $A(y)$: a single line of n_y values.
- **3D spatiotemporal** $A(\text{tr1}, \text{tr2}, t)$: `tr1` fastest-varying, then `tr2`, time slowest. $n_{\text{tr1}} \times n_{\text{tr2}} \times n_t$ values.
- **3D static spatial** $A(\text{tr1}, \text{tr2})$: `tr1` fastest-varying. $n_{\text{tr1}} \times n_{\text{tr2}}$ values.

All grids must be uniform; EPOCH reconstructs the axes from the declared bounds and point counts (`np.linspace` on the Python side). Amplitude values should be normalised to the range $[0, 1]$; EPOCH multiplies by the peak field derived from `intensity_w_cm2` (or `amp`). Phase values are in radians and are used directly — EPOCH adds no constant offset.

Interpolation is linear in every axis (bilinear for 2D spatiotemporal and 3D static, trilinear for 3D spatiotemporal). On the spatiotemporal path, positions outside the declared transverse bounds return **zero** amplitude and phase; on the static paths, positions outside the file grid are **clamped** to the edge values. In either case the profile grid should comfortably cover the boundary region the laser is meant to illuminate.

3.1 Laser profile generators

The Fortran column-major layouts above correspond to NumPy arrays in default C (row-major) order with the axes reversed, written with `.tofile()` — **no transpose is needed** in either dimension:

File	NumPy shape (C order)
2D spatiotemporal	(n_t, n_y)
2D static spatial	$(n_y,)$
3D spatiotemporal	(n_t, n_{tr2}, n_{tr1})
3D static spatial	(n_{tr2}, n_{tr1})

2D example:

```
import numpy as np

N_T   = 50
N_Y   = 30
Y_MIN, Y_MAX = -10e-6, 10e-6
T_START, T_END = 0.0, 30e-15

y = np.linspace(Y_MIN, Y_MAX, N_Y)
t = np.linspace(T_START, T_END, N_T)

# meshgrid with indexing="ij" gives arr[t_index, y_index]
Tg, Yg = np.meshgrid(t, y, indexing="ij")

# --- fill amplitude (values in [0, 1]) ---
t0, tau = 15e-15, 5e-15
w0 = 3e-6
amplitude = np.exp(-((Tg - t0) / tau)**2) * np.exp(-(Yg / w0)**2)

assert amplitude.shape == (N_T, N_Y)
amplitude.astype(np.float64).tofile("amplitude.dat")

# --- fill phase (values in radians) ---
phase = np.zeros((N_T, N_Y)) # flat phase for a plane wave
phase.astype(np.float64).tofile("phase.dat")
```

3D example (for an x_min laser, so tr1 = y and tr2 = z):

```
import numpy as np

N_Y, N_Z, N_T = 48, 40, 16          # n_tr1, n_tr2, n_t in the deck
Y_MIN, Y_MAX = -8e-6, 8e-6
Z_MIN, Z_MAX = -8e-6, 8e-6
T_START, T_END = 0.0, 60e-15

y = np.linspace(Y_MIN, Y_MAX, N_Y)
z = np.linspace(Z_MIN, Z_MAX, N_Z)
t = np.linspace(T_START, T_END, N_T)

# indexing="ij" gives arr[t_index, z_index, y_index] = (n_t, n_tr2, n_tr1)
Tg, Zg, Yg = np.meshgrid(t, z, y, indexing="ij")

t0, tau = 30e-15, 10e-15
w0 = 3e-6
```

```

amplitude = (np.exp(-((Tg - t0) / tau)**2)
             * np.exp(-(Yg**2 + Zg**2) / w0**2))

assert amplitude.shape == (N_T, N_Z, N_Y)
amplitude.astype(np.float64).tofile("amplitude.dat")

phase = np.zeros((N_T, N_Z, N_Y))
phase.astype(np.float64).tofile("phase.dat")

```

4 Phase Convention

EPOCH injects the field as:

$$E \propto \text{profile} \cdot \sin(\omega t + \text{phase}) \quad (1)$$

where ωt (the carrier phase) is accumulated internally by EPOCH. The phase file must therefore contain only the slowly varying envelope phase, not the carrier. This applies identically in 2D and 3D.

When using a complex envelope from LASY (where the physical field is $E = \text{Re}[E \cdot \exp(-i\omega_0 t)] = |E| \cdot \cos(\omega_0 t - \varphi)$), the required conversion is:

$$\text{phase_epoch} = -(\text{phase_lasy} - \phi_{\text{ref}}) - \text{gouy_phase} \quad (2)$$

where $\phi_{\text{ref}} = \angle E(\text{on axis}, t = t_{\text{peak}})$ is the on-axis phase at peak time (which removes the arbitrary global piston phase introduced by the propagator), and gouy_phase pins the result to EPOCH's carrier convention. A $\pi/2$ offset converts $\cos \rightarrow \sin$ but as a constant it does not affect field shape — the sign and reference subtraction are what matter physically.

5 Two-Channel Polarisation

A single custom-file laser block drives one transverse field component. To inject a field with independent control of *both* transverse components at a boundary — for example to synthesise an elliptically or circularly polarised drive, or any polarisation state not reducible to a single scalar profile — add a second custom-file laser block on the *same* boundary with `pol` rotated by 90° relative to the first. EPOCH's `pol` angle selects which pair of transverse field components (e.g. `Ey` and `Ez` for an `x_min` laser) a laser block drives, so `pol = 0` and `pol = 90` address the two components independently, each from its own file:

```

begin:laser
  boundary = x_min
  ...
  pol = 0
  profile_data_file = amplitude_H.dat  # drives Ey
  ...
end:laser

begin:laser
  boundary = x_min
  ...
  pol = 90
  profile_data_file = amplitude_V.dat  # drives Ez
  ...

```

```
end:laser
```

The loaded amplitude and phase data live on the `laser_block` instance itself rather than in a shared module-level variable, so each block keeps its own copy: the second block's file cannot silently overwrite or alias the first's, even when the two grids differ in shape or resolution. (Prior to this per-laser storage, a second custom-file block on the same boundary would incorrectly reuse the first block's already-loaded data.) The `cos/sin(pol_angle)` channel split that routes each block to its transverse component is identical in every boundary routine of `epoch2d` and `epoch3d`, so the same two-block pattern works unchanged in 3D. (The two-channel configuration has been validated end-to-end in 2D; the 3D per-laser storage is the same design, built in from the start.)

6 Current Limitations

- `epoch1d` has no custom-profile support.
- All file grids must be **uniform**; non-uniform coordinate axes are not supported.
- Memory: in **3D**, each MPI rank *can* store only the slab of the spatiotemporal array covering its own patch of the laser boundary (plus a one-cell margin) rather than the full plane, with loading streamed one time-slice at a time. This is now the default behaviour for a static domain decomposition: `use_pre_balance` defaults to `T` in `epoch3d` and triggers a one-off startup load balance that can move a rank's local patch, so the file load for a spatiotemporal laser is deferred past that rebalance and windowed against the settled domain instead of being sized too early. Per-rank cost is then roughly $(\text{local patch fraction}) \times n_{\text{tr1}} \times n_{\text{tr2}} \times n_t \times 8$ bytes. Continuous dynamic load balancing (`use_balance = T`) keeps the same per-rank saving for the whole run rather than only until the first rebalance: the window is re-derived and the file reloaded after every genuine mid-run redistribution, at the same per-event cost as the initial load.

The full plane is still stored on every rank, unconditionally, for a *moving injection window* on a *y/z*-boundary laser (its *tr1* axis is *x*, which the window shifts as the simulation window moves). This is deliberately retained rather than windowed: unlike domain rebalancing, the window's motion is not tied to a discrete, all-ranks-agree event such an update could hook into, so a safe local-slab alternative does not exist yet for this case. In **2D** the (small) full array is always broadcast to every rank.

7 Testing

Both `epoch2d` and `epoch3d` have an automated regression suite for this feature, integrated into the existing `tests/ nose/ unittest` framework already used by `scripts/run-tests.py` (the same runner CI exercises for the rest of the test suite). The suite lives at `tests/test_custom_laser.py`, with fixture decks and per-case working directories under `tests/custom_laser/`. It began as a port of the ad hoc diagnostic scripts used to validate this feature during development, rewritten as proper assertions against empirically-derived thresholds rather than printed numbers for manual inspection.

7.1 Running the tests

From the repository root:

```
./scripts/run-tests.py epoch3d custom_laser # or epoch2d
```

builds `epoch<N>d` and runs the suite via `nose`. Equivalently, once built, from `epoch<N>d/`:

```
nosetests tests.test_custom_laser
```

Each test class removes its own generated fixtures and simulation output (`*.sdf`, `deck.status`, `profile.dat`, ...) in `tearDownClass`; set `NOCLEAN=1` in the environment to keep them for post-mortem inspection. `./scripts/run-tests.py epoch<N>d --clean` also works, via a `makefile` in `tests/custom_laser/` that fans out to each case directory, matching the convention already used by `tests/maxwell_solvers` and `tests/custom_stencils`.

7.2 Coverage

- **y_min boundary:** a y_min-boundary spatiotemporal laser (only x_min/spatiotemporal and z_min/static were covered ad hoc before), including that boundary's own source1→field-component routing.
- **Two-laser independence:** two custom-file blocks on the same boundary with orthogonal pol, checking that neither aliases the other's loaded data – the bug that the per-laser storage change fixed.
- **Interpolation accuracy:** a separable $g(t)h(\text{transverse})$ profile lets the interpolated field ratio be checked against the analytic ratio without needing to know the carrier phase.
- **Edge behaviour:** static (clamped) versus spatiotemporal (zero-outside-bounds) sampling at and beyond the declared file bounds.
- **Phase override:** `use_phase_from_file` combined with, and without, an analytic phase tilt.
- **File-size abort:** the byte-count check in `load_binary_file` rejects a profile file whose size does not match the declared grid, for each declared dimension independently.
- **Alias errors** (epoch3d only): the propagation-axis self-alias check on all four transverse boundaries (`y_min`, `y_max`, `z_min`, `z_max`), not just `x_min`.
- **Dynamic balance re-slab** (epoch3d only): re-derives and reloads the per-rank file-slab window under `use_balance` after a runtime domain redistribution.

epoch3d runs 13 tests across these groups, epoch2d runs 10 (it has no alias-error or dynamic-balance-re-slab groups, since neither feature exists in 2D). epoch1d has no custom-profile support and is untested.